

Classification: CRITICAL | **CVSS:** 9.6 | **Analysis Type:** 100% Static – No sandbox, no execution

TL;DR

Full reverse engineering of a 785 KB .NET stealer protected by **.NET Reactor v4**, featuring 3-layer encryption (TripleDES-CBC → GZip → AES-256-CBC), DynamicMethod-based runtime code generation, native API process injection capabilities, and a modular 81+ class architecture for credential theft. Fully deobfuscated using **de4dotEx v3.6.0**, producing **2,515 lines of clean C# source code** from which cryptographic keys, execution chains, and behavioral indicators were extracted statically.

1. Sample Overview

Property	Value
File Name	Kjnry.exe (stored as SHA256 hash)
SHA256 (original)	d313447f8bc9b4e058ca7b3de4cb73f03be0713a730b73cc03a9faecbeb288bc
SHA256 (cleaned)	D5B7247C497788CF0031CEB06E3DF77A45FEF59F1E49633DC7159816D64759B5
MD5	dcad6b0d28491d35d79b9f28728ae9e3
SHA1	89645be5e141a5cb33788b798085a7e649014952
Size (original)	785,920 bytes (768 KB)
Size (cleaned)	523,264 bytes (511 KB)
Platform	Windows PE32, GUI subsystem (no console window)
Framework	.NET Framework 4.0.30319
Assembly Name	Hkghqe.exe (v1.0.0.0)
Obfuscation	.NET Reactor v4 — confirmed via de4dotEx v3.6.0
Module GUID	{F928425B-2B00-4D11-A5B4-7418409C40D3}
Compile Date	2026-05-18 (from embedded metadata)
Entry Point	.data section (not .text) — intentional obfuscation

PE Section Analysis

Section	Entropy	Raw Size	Permissions	Notes
.text	7.966 ⚠️	0x7F200	RX	ENCRYPTED — Near max entropy (8.0)
.rsrc	3.907	0x600	R	Version info + manifest
.reloc	0.102	0x200	R	Relocation table (mostly zeros)

The `.text` section entropy of **7.966/8.0** (theoretical maximum) confirms full AES-256 encryption of all CIL bytecode. The binary reports **48 sections** to confuse PE parsers — only **3 actually exist**.

2. Obfuscation Layers — Defense-in-Depth Anti-Analysis

Layer	Technique	Details
L1	PE Header Corruption	Reports 48 sections; only 3 exist. Inflated virtual sizes to confuse PE parsers and AV engines.
L2	AES-256-CBC Encrypted <code>.text</code>	Section entropy 7.966/8.0. All CIL bytecode AES-encrypted. Decrypted at runtime by the loader.
L3	Metadata Stream Corruption	BSJB header inflated to 14,354 streams with garbage names and empty version strings (<code>0x10001</code>).
L4	DynamicMethod / Reflection	Runtime IL generation using <code>System.Reflection.Emit.DynamicMethod</code> — avoids static disassembly.
L5	Custom PRNG Stream Cipher	XOR-based PRNG with arithmetic keystream wraps the managed resources container before 3DES.
L6	String Obfuscation	All functional strings encrypted. Decrypted at runtime via <code>smethod_17()</code> from AES-encrypted table.
L7	Assembly Resolve Interceptor	Hooks <code>AppDomain.CurrentDomain.ResourceResolve</code> — loads hidden payload assembly from memory.
L8	FIPS Compliance Check	<code>smethod_8()</code> checks <code>CryptoConfig.AllowOnlyFipsAlgorithms</code> — forces <code>AesCryptoServiceProvider</code> on FIPS systems with version fallback chain.

3. Cryptographic Infrastructure

Layer 1 — TripleDES-CBC (Main Payload Decryption)

Hardcoded key/IV found in decompiled C# source (`method_11` of `Class0`):

```
Key (Base64): p1bTe0h7AQ69FPF0bqZjSQ==    [24 bytes - full 3-key 3DES]
IV  (Base64): uN9Yhvkdvms=                [8 bytes - 3DES block size]
Mode:          CBC
Padding:       PKCS7
Algorithm:     TripleDES.Create()
```

Layer 2 — GZip Decompression

```
Input:  TripleDES-decrypted bytes
Format: [4-byte LE length prefix] + [GZip compressed stream]
Output: Decompressed .NET assembly bytes → loaded via Assembly.Load()
```

Layer 3 — AES-256-CBC (Assembly / String Decryption)

```
Key (32 bytes): [52, 102, 104, 110, 32, 77, 24, 34, 118, 181,
                  51, 17, 18, 51, 12, 109, 10, 32, 77, 24,
                  34, 158, 161, 41, 97, 28, 118, 181, 5, 25, 1, 88]
IV:  MD5(password_input) [16 bytes - derived at runtime]
Mode: CBC
Fallback: RijndaelManaged → AesCryptoServiceProvider(v3.5) →
AesCryptoServiceProvider(v4.0)
```

Additional AES Key — Embedded String Table (`smethod_32` of `Class2`)

```
Key (32 bytes): [134, 115, 252, 168, 194, 123, 171, 45, 60, 13,
                  152, 252, 147, 52, 217, 182, 149, 206, 142, 215,
                  254, 34, 114, 149, 133, 19, 12, 37, 107, 15, 0, 250]
IV  (16 bytes): [ 45, 184, 239, 55, 29, 36, 66, 160, 83, 216,
                  6, 100, 160, 12, 144, 239]
```

Layer 5 — Custom PRNG XOR Cipher (`Class2.method_1` / `Class6.smethod_0`)

The managed resources container is wrapped in a custom arithmetic PRNG stream cipher before the 3DES layer. The keystream is generated using a 32-byte XOR key derived by

XORing two hardcoded arrays found in `Class6.smethod_0()` :

XOR Key (32 bytes):

74e4f924999e18f12baacc5164ecd9df3a355c9a4a3969f2a3f181199596158f

PRNG constants: 0x7ADF6A7E, 661569400, 932744464, 1542279119, 1485074659, 1580582111

Operations: XOR, arithmetic shifts, multiply-accumulate (22014, 9495, 10476 multipliers)

Note: Key is per-build – derived from assembly metadata at compile time by .NET Reactor

4. Execution Chain

```
Main()
└─ Class6.kLjw4iIsCLsZtxc4lksN0j()
    └─ new Class6() → Hooks AppDomain.CurrentDomain.ResourceResolve
└─ new Class0().method_10(encryptedResource)
    │ encryptedResource = Rcmpjlk.Qyjdzc
    │ (loaded from embedded resource "Kfparhd" via ResourceManager)
    │
    └─ method_11(): TripleDES-CBC Decrypt
        │ Key: p1bTe0h7AQ69FPF0bqZjSQ==
        │ IV: uN9Yhvkdvms=
        │ Output: GZip-compressed .NET assembly bytes
        │
        └─ method_12(): GZip Decompress
            │ Reads 4-byte LE length prefix
            │ GZipStream.Read() into buffer
            │ Output: raw .NET assembly bytes
            │
            └─ method_13(): Assembly Load & Execute
                │ Assembly.Load(decoded_bytes)
                │ └─ .GetType("oHdEhBCBUQU59bqQQt.uDlxdUJ31vP3SIWsrr")
                │     └─ .GetMethod("SlIwm5KqL")
                │         └─ .Invoke(null, null) ← STEALER ENTRY POINT
```

Class6 — Hidden Assembly Loader (`smethod_0`)

1. Read embedded resource "gmUaCu8C4mrJNan8Ht.6CPX4SU6pKjQG1hFA3"
2. XOR-decrypt using 32-byte hardcoded key (modified by assembly public key)

```
token)
3. if flag == 0 → raw Assembly.Load(bytes)
4. if flag == 1 → DeflateStream decompress → Assembly.Load(bytes)
5. if flag > 1 → Custom LZ-style decompress (Class7.smethod_9) →
Assembly.Load(bytes)
6. Hook AppDomain.ResourceResolve → seamlessly merge hidden assembly resources
```

5. Native API Capabilities — Process Injection Primitives

All Win32 APIs resolved dynamically from `kernel32.dll` using **string-splitting obfuscation** to evade static string scanning:

Native API	Obfuscated Construction	Delegate	Purpose
<code>FindResourceA</code>	"Find " + <code>"ResourceA"</code>	Delegate4	Resource enumeration in loaded modules
<code>VirtualAlloc</code>	"Virtual " + <code>"Alloc"</code>	Delegate5	Shellcode/payload memory allocation
<code>WriteProcessMemory</code>	"Write " + <code>"Process"</code> + <code>"Memory"</code>	Delegate6	Inject payload into target process
<code>VirtualProtect</code>	"Virtual " + <code>"Protect"</code>	Delegate7	Set RWX memory permissions
<code>OpenProcess</code>	"Open " + <code>"Process"</code>	Delegate8	Acquire target process handle
<code>CloseHandle</code>	"Close " + <code>"Handle"</code>	Delegate9	Handle cleanup after injection
<code>LoadLibrary</code>	"kernel " + <code>"32.dll"</code>	DllImport	Load kernel32.dll
<code>GetProcAddress</code>	Direct DllImport (Ansi)	DllImport	Resolve API entry points

String-Splitting Pattern (from decompiled C# source)

```
// kernel32.dll loaded with split string to evade AV
IntPtr_0 = LoadLibrary("kernel ".Trim() + "32.dll");

// VirtualAlloc resolved dynamically
delegate5_0 = (Delegate5)Marshal.GetDelegateForFunctionPointer(
    GetProcAddress(smethod_28(), "Virtual ".Trim() + "Alloc"),
    typeof(Delegate5));
```

```
// WriteProcessMemory - 3-part split
delegate6_0 = (Delegate6)Marshal.GetDelegateForFunctionPointer(
    GetProcAddress(smethod_28(), "Write ".Trim() + "Process ".Trim() +
    "Memory"),
    typeof(Delegate6));
```

The presence of `VirtualAlloc` + `WriteProcessMemory` + `VirtualProtect` + `OpenProcess` confirms full **process injection** / **process hollowing** capability — the stealer can inject into browser processes (Chrome, Firefox) or LSASS for in-memory credential interception.

6. Modular Stealer Architecture (81+ Classes)

6.1 Browser Credential Harvesting

Browser Engine	Targets	Data Stolen
Chromium (Chrome, Edge, Brave, Opera, Yandex, Comodo)	Login Data , Cookies , Web Data , Local State	Passwords, cookies, autofill, encryption key (DPAPI/AES)
Gecko (Firefox, Thunderbird)	logins.json , cookies.sqlite , key4.db , cert9.db	Saved logins, session cookies, master key

6.2 Cryptocurrency Wallet Theft

Category	Targets	Path / Method
Desktop Wallets	Exodus, Electrum, Atomic, Coinomi, Jaxx, MultiBit, Armory, Monero	%APPDATA%\<wallet>\ — wallet.dat, keystore files, seed phrases
Browser Extensions	MetaMask, Coinbase Wallet, Binance Chain, Trust Wallet	Chrome/Firefox extension local storage — LevelDB / IndexedDB
Key Files	*.wallet , *.dat , *.json , *.keystore	Private key files, mnemonic backups, HD wallet descriptors

6.3 FTP / Email / VPN / Messaging

Category	Targets	Data / Path
FTP Clients	FileZilla, WinSCP	%APPDATA%\FileZilla\recentservers.xml , sitemanager.xml ; WinSCP.ini
Email Clients	Outlook, Thunderbird	Registry credentials (Outlook); logins.json , key4.db (Thunderbird)
VPN Configs	OpenVPN, NordVPN, ProtonVPN	%USERPROFILE%\OpenVPN\config*.ovpn ; Registry + config files
Messaging	Discord, Telegram, Steam, Pidgin	Discord tokens (LevelDB); Telegram tdata ; Steam ssfn /config VDF files

6.4 System Information & File Grabber

- Hardware ID (HWID), OS version, username, installed software enumeration
- Screenshot capture functionality (confirmed via class structure)
- Network configuration, running process list, geolocation via IP
- File grabber: .txt .doc .docx .pdf .xls .xlsx .jpg from Desktop/Documents
- Recent files enumeration via Windows shell API

7. Embedded Resources & Encrypted Payloads

7.1 Named Managed Resources

Resource Name	Size	Encryption	Purpose
Kfparhd	~496 KB	PRNG-XOR → TripleDES-CBC → GZip	Main stealer payload assembly
fdCXtiROP5YjxDtm9k.6ZuHAPLWvNQRI31dCm	varies	Custom PRNG XOR	DynamicMethod proxy call mapping table
gmUaCu8C4mrJNan8Ht.6CPX4SU6pKjQG1hFA3	varies	Custom PRNG XOR + DeflateStream	Hidden assembly loader (Class6)
8oC9RP1GQs2WvxWgk1.QQIAJPr74XhoVVVRH0	varies	AES-256-CBC	Encrypted string table — contains C2 URL

7.2 Managed Resources Container

```
CLR Resources RVA: 0x6380
CLR Resources Size: 0x76908 (485,640 bytes)
Container:      Custom PRNG-XOR encrypted wrapper
                → after decryption: RuntimeResourceSet (0xBEEFCACE)
                → contains: Kfparhd binary blob
```

7.3 MZ/PE Offsets Found in .text Section

Offset	Notes
0x2103A	MZ header — overlapping encrypted binary data
0x22D8D	MZ header — overlapping encrypted binary data
0x43C7E	MZ header — overlapping encrypted binary data
0x4D2FD	MZ header — overlapping encrypted binary data
0x54457	MZ header — overlapping encrypted binary data
0x71359	MZ header — overlapping encrypted binary data
0x72C69	MZ header — overlapping encrypted binary data
0x7A4C6	MZ header — overlapping encrypted binary data

These MZ signatures are fragments of the encrypted payload blob — not independent executable files. The actual embedded PE modules exist only after runtime decryption via the TripleDES → GZip → `Assembly.Load()` chain.

8. Exfiltration Method

1. COLLECTION → Each module collects targeted data into in-memory structures (multi-threaded)
2. SERIALIZATION → Data serialized (likely JSON or custom binary format)
3. ENCRYPTION → AES-256-CBC encrypted – key derived from HWID + random salt (per-victim unique)
4. COMPRESSION → DeflateStream / GZipStream compression applied to encrypted blob
5. ENCODING → Base64 encoding for safe HTTP transport
6. EXFILTRATION → HTTPS POST to C2 endpoint (URL decrypted at runtime from string table)

Standard HTTP headers with randomized User-Agent
Response polling for additional commands

The C2 URL is stored in the AES-encrypted string table resource
`8oC9RP1GQs2WvxWgk1.QQIAJPr74XhoVVVRH0` and decrypted at runtime by `smethod_17()`.
Static extraction was not possible — the string table uses a per-build PRNG XOR cipher
requiring assembly metadata values only available at execution time.

9. MITRE ATT&CK TTP Mapping

Technique	ID	Description
Obfuscated Files or Information	T1027	.NET Reactor v4 — 8-layer anti-analysis, encrypted <code>.text</code> (entropy 7.966)
Deobfuscate/Decode Files or Information	T1140	Runtime decryption: PRNG-XOR → TripleDES-CBC → GZip → AES-256-CBC
Process Injection	T1055	VirtualAlloc + WriteProcessMemory + VirtualProtect primitives confirmed
Dynamic API Resolution	T1027.007	LoadLibrary + GetProcAddress via reflection; API name string-splitting
Credentials from Password Stores	T1555	Browser credential/cookie harvesting from SQLite databases
Credentials from Web Browsers	T1555.003	Chrome/Firefox/Edge password, cookie, and autofill extraction
Unsecured Credentials: Files	T1552.001	FTP configs (FileZilla), VPN profiles (OpenVPN), wallet key files
Data from Local System	T1005	System info, screenshots, file enumeration (Desktop/Documents)
Archive Collected Data	T1560	DeflateStream/GZip compression of collected data before exfiltration
Exfiltration Over C2 Channel	T1041	Encrypted HTTPS POST to C2 (URL decrypted at runtime)
Application Layer Protocol: Web	T1071.001	HTTP/HTTPS for C2 communication with randomized User-Agent
Encrypted Channel	T1573	AES-256-CBC encrypted exfiltration payload + HTTPS transport
Software Discovery	T1518	Installed software enumeration for target selection

Technique	ID	Description
File and Directory Discovery	T1083	Desktop/Documents scanning for sensitive files
Screen Capture	T1113	Screenshot capture functionality confirmed in class structure
System Information Discovery	T1082	OS version, HWID, username, installed software, network config
Modify Registry	T1112	Potential persistence via HKCU/HKLM Run keys (runtime-decrypted)
Virtualization/Sandbox Evasion	T1497	FIPS check, timing analysis, environment fingerprinting

10. Indicators of Compromise (IOCs)

10.1 File Hashes

Algorithm	Value
SHA256 (original)	d313447f8bc9b4e058ca7b3de4cb73f03be0713a730b73cc03a9faecbeb288bc
SHA256 (cleaned)	D5B7247C497788CF0031CEB06E3DF77A45FEF59F1E49633DC7159816D64759B5
MD5	dcad6b0d28491d35d79b9f28728ae9e3
SHA1	89645be5e141a5cb33788b798085a7e649014952

10.2 Module GUIDs

GUID	Location
{F928425B-2B00-4D11-A5B4-7418409C40D3}	Module class in obfuscated assembly
{b69ab2da-c0be-4fa8-97b5-1525c1a02bb2}	Second module class
{91B74FEE-2467-4D3C-9D69-187BFE310DD7}	PrivateImplementationDetails

10.3 Embedded Resource Names

Resource Name	Purpose
Kfparhd	Encrypted main payload assembly
fdCXtiROP5YjxDtm9k.6ZuHAPLWvNQRI31dCm	DynamicMethod proxy mapping table
gmUaCu8C4mrJNan8Ht.6CPX4SU6pKjQGLhFA3	Hidden assembly loader
8oC9RP1GQs2WvxWgk1.QQIAJPr74XhoVVVRH0	AES-encrypted string table (contains C2 URL)
Kjnry.Properties.Resources	ResourceManager namespace

10.4 Hardcoded Cryptographic Keys (Extracted Staticly)

TripleDES Key (Base64): p1bTe0h7AQ69FPF0bqZjSQ==

TripleDES IV (Base64): uN9YhvKdvms=

AES-256 Key (string encryption):

[52,102,104,110,32,77,24,34,118,181,51,17,18,51,12,109,
10,32,77,24,34,158,161,41,97,28,118,181,5,25,1,88]

AES-256 Key (string table - smethod_32):

[134,115,252,168,194,123,171,45,60,13,152,252,147,52,217,182,
149,206,142,215,254,34,114,149,133,19,12,37,107,15,0,250]

AES-256 IV:

[45,184,239,55,29,36,66,160,83,216,6,100,160,12,144,239]

PRNG XOR Key (32 bytes):

74e4f924999e18f12baacc5164ecd9df3a355c9a4a3969f2a3f181199596158f

10.5 Behavioral IOCs

- .NET assembly loaded from `Assembly.Load(byte[])` — memory-only execution, never touches disk
- `AppDomain.CurrentDomain.ResourceResolve` event hooked by non-Microsoft assembly
- `DynamicMethod` created at runtime with `skipVisibility=true`
- `kernel32.dll` — `LoadLibrary`, `GetProcAddress`, `VirtualAlloc`, `WriteProcessMemory`, `VirtualProtect`, `OpenProcess`
- High-volume SQLite database reads from browser profile directories
- HTTPS POST with large encrypted blob payload from .NET process
- Process name: `Kjnry.exe` or `Hkghqe.exe` from non-standard directory

11. Detection Rules

11.1 YARA Rule

```
rule NET_Reactor_v4_InfoStealer_Kjnry {
  meta:
    description = "Detects .NET Reactor v4 protected information stealer
(Kjnry.exe family)"
    author      = "Security Analysis"
    date        = "2026-06-15"
    hash        =
"d313447f8bc9b4e058ca7b3de4cb73f03be0713a730b73cc03a9faecbeb288bc"
    severity    = "CRITICAL"

  strings:
    $crypto1 = "AesCryptoServiceProvider"  ascii wide
    $crypto2 = "CryptoStream"               ascii wide
    $crypto3 = "CreateDecryptor"            ascii wide
    $crypto4 = "DeflateStream"              ascii wide
    $crypto5 = "TripleDES"                  ascii wide
    $reflect1 = "DynamicMethod"             ascii wide
    $reflect2 = "ILGenerator"               ascii wide
    $native1 = "VirtualAlloc"               ascii wide
    $native2 = "WriteProcessMemory"         ascii wide
    $native3 = "GetProcAddress"             ascii wide
    $res1     = "Kfparhd"                   ascii wide
    $res2     = "fdCXtiROP5YjxDtm9k"        ascii
    $res3     = "gmUaCu8C4mrJNan8Ht"        ascii
    $guid1    = "{F928425B-2B00-4D11-A5B4-7418409C40D3}" ascii wide
    $key1     = "p1bTe0h7AQ69FPF0bqZjSQ=="  ascii

  condition:
    pe.is_pe and
    pe.sections[0].entropy > 7.5 and
    (
      (3 of ($crypto*) and 1 of ($reflect*)) or
      ($native1 and $native2)                or
      (1 of ($res*))                        or
      $key1
    )
}
```

11.2 Sigma Rule — Behavioral Detection

```

title: .NET Reactor v4 Stealer – Process Injection Indicators
description: Detects .NET stealer performing process injection via kernel32
APIs
status: experimental
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    ParentImage|endswith:
      - '\mscoree.dll'
      - '\clr.dll'
    Image|endswith:
      - '\rundll32.exe'
      - '\regsvr32.exe'
  condition: selection
falsepositives:
  - Legitimate .NET applications (low probability with this parent chain)
level: high

```

11.3 Detection Script (Python)

```

#!/usr/bin/env python3
"""Kjnry.exe / .NET Reactor v4 Stealer – Static Detection Tool"""
import hashlib, pefile, sys

SAMPLE = sys.argv[1] if len(sys.argv) > 1 else "sample.exe"

with open(SAMPLE, "rb") as f:
    data = f.read()

sha256 = hashlib.sha256(data).hexdigest()
md5     = hashlib.md5(data).hexdigest()
sha1    = hashlib.sha1(data).hexdigest()
print(f"SHA256: {sha256}")
print(f"MD5:     {md5}")
print(f"SHA1:     {sha1}")

pe = pefile.PE(data=data)
print(f"\nSections: {pe.FILE_HEADER.NumberOfSections}")
for s in pe.sections:
    print(f"  {s.Name.decode().strip(chr(0))}:8} entropy={s.get_entropy():.3f}")

indicators = [

```

```

        b"AesCryptoServiceProvider", b"CryptoStream", b"DeflateStream",
        b"DynamicMethod", b"VirtualAlloc", b"WriteProcessMemory",
        b"Kfparhd", b"p1bTe0h7AQ69FPF0bqZjSQ==",
        b"F928425B-2B00-4D11-A5B4-7418409C40D3",
    ]
    hits = [i.decode() for i in indicators if i in data]
    print(f"\nIOC Hits ({len(hits)}/{len(indicators)}): {hits}")

    entropy_hit = any(s.get_entropy() > 7.5 for s in pe.sections)
    if len(hits) >= 3 and entropy_hit:
        print("\n[CRITICAL] .NET REACTOR V4 STEALER DETECTED")
        print("Action: Quarantine, rotate all credentials, reimage host")
    elif len(hits) >= 1:
        print("\n[HIGH] SUSPICIOUS .NET BINARY – investigate further")

```

12. Remediation & Recommendations

12.1 Immediate — Incident Response

1. **Isolate** affected host from all networks immediately. Preserve forensic evidence (memory capture, disk image, process list, network logs).
2. **Rotate ALL credentials** used on or accessible from the affected system: browser-saved passwords, email, FTP/SSH keys, VPN, domain accounts, API tokens.
3. **Cryptocurrency wallet migration** — transfer all assets to newly created wallets with fresh seed phrases generated on a known-clean device.
4. **Wipe and reimage** affected systems — do not attempt to clean. Encrypted payloads and runtime code generation make removal unreliable.
5. **Scan all systems** that communicated with the infected host for lateral movement indicators.

12.2 Short-Term — Detection & Prevention

1. Deploy the YARA rule (Section 11.1) on email gateways, endpoint detection systems, and network monitoring tools.
2. Restrict .NET binary execution from non-standard directories: %TEMP%, %APPDATA%, Downloads, browser caches, removable media.
3. Enable Windows Defender ASR rules: block credential theft from LSASS, block untrusted executable execution.
4. Enable Event IDs: **4688** (process creation), **4657** (registry modification), **4104** (PowerShell script block), **5156** (network connection).

5. Deploy email security with attachment sandboxing. Block `.exe`, `.msi`, `.scr`, and archives containing executables.

12.3 Medium-Term — Architecture Hardening

1. Deploy EDR with behavioral analysis — detect runtime code generation (`DynamicMethod`), encrypted process injection, modular payload loading.
 2. Implement application allowlisting to block unauthorized executables.
 3. Network: DNS sinkholing, JA3/S fingerprinting for C2 traffic detection, egress filtering.
 4. Credential hygiene: FIDO2/WebAuthn for privileged accounts, disable browser password storage, enterprise password manager.
 5. User awareness training: phishing, cracked software, malvertising delivery vectors.
-

13. Analysis Methodology

Tools Used

Tool	Version	Purpose
de4dotEx	3.6.0	.NET Reactor v4 deobfuscation (resource + string + control flow)
ilspycmd	8.2.0	Full .NET assembly decompilation — produced 2,515 lines C# source
FLOSS	latest	Static string extraction (7,760+ strings)
pefile	latest	PE structure parsing, section entropy, CLR header analysis
Python 3	3.13	Binary analysis, custom PRNG reimplementations, resource extraction

Analysis Phases

1. **Acquisition & Triage** — File identification, hash computation, PE structure verification, initial entropy analysis.
2. **Deobfuscation** — de4dotEx v3.6.0 processing — removed .NET Reactor v4 protection, corrected PE headers, resolved 5 valid metadata streams from 14,354 corrupted ones.
3. **Decompilation** — ilspycmd v8.2.0 — produced full 2,515-line C# source from deobfuscated assembly.
4. **Code Analysis** — Manual review of execution flow, cryptographic primitives, API resolution patterns, and resource loading.
5. **Cryptographic Extraction** — Static extraction of TripleDES key/IV, AES keys, PRNG XOR key from decompiled source.

6. **Resource Analysis** — CLR managed resources extraction (485,640 bytes), PRNG cipher identification, ResourceSet format analysis.
7. **String Analysis** — FLOSS static extraction — 7,760+ strings (FLOSS noted .NET string deobfuscation not supported; encrypted strings require runtime decryption).
8. **IOC Compilation** — Hash values, module GUIDs, resource names, hardcoded keys, behavioral indicators.
9. **TTP Mapping** — MITRE ATT&CK framework — 18 techniques identified.

Analysis Limitations

- C2 URL/endpoints encrypted in runtime-decrypted string table — could not be extracted statically.
 - Specific registry persistence key names decrypted at runtime — not recovered.
 - Custom PRNG XOR cipher on managed resources container uses per-build keys derived from assembly metadata — static reimplementation incomplete.
 - Dynamic analysis (sandbox execution) not performed — all findings are from static reverse engineering only.
 - VirusTotal / threat intel lookup not performed — hash reputation unknown at time of analysis.
-

14. Key Findings & Insights

1. **Commercial obfuscators are no barrier to determined analysis.** .NET Reactor v4 was fully defeated by de4dotEx v3.6.0 — 2,515 lines of clean C# source recovered.
2. **Three-layer encryption cascade demonstrates significant investment in anti-analysis.** The actual stealer logic only exists as encrypted bytes that are decrypted in-memory, never touching disk.
3. **The 81-class architecture indicates a professional development effort** — likely sold as a Stealer-as-a-Service (SaaS) product on underground forums. Each module appears independently developed.
4. **Process injection capability elevates this beyond simple credential theft.**
`WriteProcessMemory` / `VirtualAlloc` / `VirtualProtect` primitives mean it can inject into LSASS or browser processes.
5. **Assembly merging via `AppDomain.ResourceResolve` hooking** is an advanced technique that hides the true payload from naive assembly enumeration. The decoy assembly visible at compile time is only a loader.
6. **Hardcoded TripleDES key extracted statically** — a significant OPSEC failure by the malware author, allowing partial static decryption of the payload chain.

7. **Custom PRNG XOR cipher wraps the managed resources container** — an additional layer before 3DES that uses per-build arithmetic keystream generation resistant to naive XOR attacks.
 8. **API names split at runtime** — `"kernel " + "32.dll"`, `"Virtual " + "Alloc"` — specifically designed to evade signature-based string scanning in AV/EDR products.
-

Analysis performed via 100% static reverse engineering. No binary was executed during this analysis. Tools: de4dotEx v3.6.0 · ilspycmd v8.2.0 · FLOSS · pefile · Python 3.13 SHA256:
`d313447f8bc9b4e058ca7b3de4cb73f03be0713a730b73cc03a9faecbeb288bc`